Práctica 1 · Recibir datos mientras el programa trabaja

En la práctica 0 el cliente solo hacía una cosa: esperar tramas y mostrarlas. Un programa real tiene más trabajo (atender al operario, calcular, mover un robot) y, mientras lo hace, los datos siguen llegando. En esta práctica vas a ver qué pasa con esos datos y cómo organizar el programa para que no se queden viejos ni se pierdan sin que nadie se entere.

Objetivos de la práctica

- Medir la **edad** de un dato: cuánto hace que se envió cuando tu programa lo usa.
- Comprobar que TCP entrega un flujo de bytes, no mensajes.
- Separar en dos **hilos** una tarea que bloquea el programa y la recepción de datos, y compartir el último dato de forma segura con un **candado**.
- Atender a varios clientes a la vez, con un hilo por cliente.
- Detectar con un **watchdog** que un equipo ha dejado de enviar, y distinguir un equipo desconectado de uno colgado.

Material: un PC con Python 3.10 o posterior y VS Code con la extensión Python de Microsoft, como en la práctica 0.

> **Atención**
>
> En esta práctica **no se entrega código**. Escribirás cada programa a mano a partir de los listados del Anexo A, en el momento en que lo pida su paso. Escribirlo obliga a leer cada línea: es parte del aprendizaje.

Archivo	Listado	Lo escribes en	Líneas
`e1_servidor_tiempo.py`	A.1	Paso 1	44
`e1_cliente_tiempo.py`	A.2	Paso 2	54
`e1_cliente_bytes.py`	A.3	Paso 5	32
`e2_operario_sin_hilo.py`	A.4	Paso 10	43
`e2_operario_con_hilo.py`	A.5	Paso 11	101

Archivo	Listado	Lo escribes en	Líneas
`e3_supervisor.py`	A.6	Paso 14	128
`e3_dispositivo.py`	A.7	Paso 14	26

Cómo trabajar

> **En el laboratorio**
>
> **Paso 0 · Preparar la carpeta.** Crea una carpeta `practica1` y ábrela en VS Code con *Archivo → Abrir carpeta…* (*File → Open Folder…*). Crea dentro cada archivo con el nombre exacto de la tabla.

Cada programa se puede lanzar de dos formas. Las usarás las dos a la vez, porque casi siempre habrá al menos dos programas en marcha:

Forma	Cómo se lanza	Cómo se detiene	Para qué
En un terminal	*Terminal → Nuevo terminal* y `python nombre.py`	Ctrl + C	Programas que solo tienen que estar en marcha
En depuración	Con el archivo abierto, F5 → *Python Debugger → Python File*	Mayús + F5	El programa en el que vas a poner puntos de interrupción

Si ya hay una sesión de depuración abierta y pulsas F5 en otro archivo, VS Code puede preguntar si quieres iniciar otra instancia: acepta. Los nombres exactos de menús y opciones pueden variar con la versión de VS Code y su idioma.

Los números de línea que se citan en el enunciado son los de los listados del Anexo A. Si al escribir añades o quitas líneas en blanco, los tuyos no coincidirán exactamente.

1.1 Qué vamos a montar

La práctica tiene tres ejemplos. Cada uno deja a la vista un problema que resuelve el siguiente:

Ejemplo	Qué hay en marcha	Qué vas a descubrir
1	Un servidor que envía la hora cada 0,5 s y un cliente que la recibe	Si el cliente está ocupado, los datos que usa son cada vez más viejos. Y si el servidor se cuelga, el cliente espera para siempre sin enterarse
2	El mismo servidor y un programa que espera a que el operario pulse Enter	Cómo seguir recibiendo mientras el programa está bloqueado: un hilo aparte
3	Un supervisor y varios dispositivos que le envían su hora	Cómo atender a varios a la vez y cómo detectar al que se ha colgado: un watchdog

Como en la práctica 0, todo se ejecuta en tu PC y los programas se comunican por la interfaz de *loopback* (127.0.0.1). Al final, si hay tiempo, lo repetirás entre dos PC.

El mensaje: la hora en 8 bytes

Los tres ejemplos usan el mismo mensaje, el más sencillo posible: la hora a la que se envía. Es lo único que hace falta para medir cuánto tarda un dato en llegar y cuánto tiempo pasa hasta que tu programa lo usa.

La hora se obtiene con `time.time()`: los segundos transcurridos desde el 1 de enero de 1970, con decimales. Se envía como **real de 64 bits en big-endian**, formato `">d"` de `struct`. Por ejemplo, para las 21:00:00,25 del 6 de octubre de 2026 (hora peninsular):

Byte	0	1	2	3	4	5	6	7
Valor (hex)	`41`	`da`	`b1`	`4a`	`44`	`10`	`00`	`00`

`struct.unpack(">d", ...)` convierte esos 8 bytes en `1791306000.25`.

No hay cabecera: todos los mensajes miden lo mismo, así que el receptor sabe que cada 8 bytes empieza uno nuevo. Es una simplificación respecto a la trama del UR y de la práctica 0, donde la cabecera decía cuánto medía cada mensaje.

¿Por qué `d` y no `f`? `time.time()` vale unos 1 800 000 000. Un real de 32 bits guarda unas 7 cifras significativas (capítulo 6.1), así que no puede representar ni siquiera los segundos: `1791306000.25` empaquetado con `">f"` vuelve como `1791305984.0`, ¡16 s de error! Un real de 64 bits guarda unas 15 cifras: llega al microsegundo.

Dos medidas que usarás toda la práctica

- **Edad del dato** = hora del receptor − hora que viene en el mensaje. Es cuánto hace que se envió el dato que estás usando.
- **Intervalo** = hora de un mensaje − hora del anterior. Es cada cuánto envía realmente el emisor.

> **Idea clave**
>
> La edad del dato mezcla dos cosas: lo que tarda el mensaje en viajar y lo que tarda tu programa en leerlo. En el mismo PC el viaje dura décimas de milisegundo, así que casi toda la edad que veas será culpa de tu programa.

1.2 Ejemplo 1 · Un servidor que solo envía la hora

El servidor, paso a paso

El servidor hace el papel de un dispositivo que publica datos sin que nadie se los pida, como el robot: en cuanto alguien se conecta, le envía la hora cada medio segundo.

La configuración (líneas 7–9). Como en la práctica 0, todo lo que vas a cambiar en los experimentos está arriba, en constantes:

```
7  HOST = "127.0.0.1"   # "0.0.0.0" para aceptar conexiones de otros PC
8  PORT = 50000
9  PERIODO = 0.5        # segundos entre mensajes
```

Enviar la hora (líneas 12–20). Una vez conectado un cliente, la función `atender` repite tres cosas sin parar: empaqueta la hora, la envía y espera.

```
15      while True:
16          conexion.sendall(struct.pack(">d", time.time()))
17          enviados += 1
18          if enviados % 10 == 0:
19              print(f"{enviados} mensajes enviados")
20          time.sleep(PERIODO)
```

La línea 16 hace en una sola instrucción lo que en la práctica 0 ocupaba toda `build_message`: `time.time()` da la hora, `struct.pack(">d", ...)` la convierte en 8 bytes y `sendall` los envía. Para no llenar el terminal, solo se informa cada 10 mensajes

(líneas 18–19).

Esperar clientes (líneas 23–37). El arranque es el de la práctica 0: `socket`, `bind` y `listen`. La diferencia es el bucle de la línea 29: cuando un cliente se va, el servidor no termina, sino que vuelve a `accept()` y espera al siguiente.

```
 29          while True:
 30              print(f"Esperando un cliente en {HOST}:{PORT} ...")
 31              conexion, direccion = escucha.accept()
 32              print(f"Cliente conectado desde {direccion[0]}:
{direccion[1]}")
 33              with conexion:
 34                  try:
 35                      atender(conexion)
 36                  except (BrokenPipeError, ConnectionResetError,
ConnectionAbortedError):
 37                      print("El cliente se ha desconectado")
```

¿Cómo se entera el servidor de que el cliente se ha ido? Al intentar enviarle algo: `sendall` falla con una de las excepciones de la línea 36. Así no tendrás que relanzar el servidor cada vez que relances el cliente.

> **En el laboratorio**
>
> **Paso 1 · El servidor.** Escribe `e1_servidor_tiempo.py` (Anexo A.1). Ábrete un terminal y lánzalo con `python e1_servidor_tiempo.py`. Debe aparecer:
>
> `Esperando un cliente en 127.0.0.1:50000 ...`
>
> Déjalo en marcha: lo usarás en los ejemplos 1 y 2.

El cliente, paso a paso

El cliente se parece mucho al de la práctica 0. Reutiliza `recv_exactly` tal cual (líneas 12–20): aquí cada mensaje mide 8 bytes, así que siempre se piden 8.

La configuración (líneas 7–9). `PROCESADO` simula que el programa hace algo con cada dato antes de leer el siguiente. Empieza valiendo 0.

El bucle principal (líneas 33–43).

```
 33          while True:
 34              (t_servidor,) = struct.unpack(">d", recv_exactly(sock, 8))
 35              n += 1
 36              edad = time.time() - t_servidor
 37              texto = f"n = {n:4d}   hora del servidor
{hora_legible(t_servidor)}"
 38              texto += f"   edad {edad * 1000:8.1f} ms"
 39              if anterior is not None:
 40                  texto += f"   intervalo {(t_servidor - anterior) *
1000:7.1f} ms"
 41              print(texto)
 42              anterior = t_servidor
 43              time.sleep(PROCESADO)
```

Línea	Qué hace
34	Espera los 8 bytes del siguiente mensaje y los convierte en un número. Recuerda que `unpack` siempre devuelve una tupla: por eso `(t_servidor,)`
36	Calcula la edad: hora de este PC menos hora del servidor
39–40	Calcula el intervalo con el mensaje anterior. El primer mensaje no tiene anterior
43	«Procesa» el dato. Mientras duerme, no lee

`hora_legible` (líneas 23–25) solo convierte los segundos desde 1970 en algo como `21:00:00.250`.

> **En el laboratorio**
>
> **Paso 2 · El cliente.** Escribe `e1_cliente_tiempo.py` (Anexo A.2) y lánzalo con F5, en depuración. Debes ver algo así:

```
n =    1   hora del servidor 18:17:06.206   edad      0.9 ms
n =    2   hora del servidor 18:17:06.706   edad      0.2 ms   intervalo
500.3 ms
n =    3   hora del servidor 18:17:07.206   edad      0.2 ms   intervalo
500.3 ms
n =    4   hora del servidor 18:17:07.707   edad      0.2 ms   intervalo
500.3 ms
```

En el mismo PC, un mensaje tarda **unas décimas de milisegundo** en ir de un programa a otro. Esa es la referencia para todo lo que viene después. Anota tus valores.

> **En el laboratorio**
>
> **Paso 3 · Mirar dentro.** Pon un punto de interrupción en la línea 36. Cuando el cliente se detenga, escribe en la Consola de depuración (*Debug Console*):
>
> `t_servidor` · `time.time()` · `time.time() - t_servidor`
>
> Repite la última varias veces. ¿Por qué crece? Quita el punto de interrupción (F9) y continúa (F5). Guarda en la memoria lo que pasa justo después: lo explicarás en el experimento 3.

1.3 Experimentos con la hora

Cinco experimentos con el mismo par de programas. En cada uno, **antes de ejecutar, escribe qué crees que va a pasar**. Después compruébalo.

Experimento 1 · Un cliente lento

> **En el laboratorio**
>
> **Paso 4.** En el cliente, cambia la línea 9 a `PROCESADO = 2.0` y relánzalo. Deja que muestre unas diez líneas.

```
n =    1   hora del servidor 18:17:09.207   edad      0.1 ms
n =    2   hora del servidor 18:17:09.708   edad   1500.3 ms   intervalo
500.2 ms
n =    3   hora del servidor 18:17:10.208   edad   3000.4 ms   intervalo
500.3 ms
n =    4   hora del servidor 18:17:10.708   edad   4500.6 ms   intervalo
500.2 ms
```

El servidor envía dos mensajes por segundo y el cliente lee uno cada dos segundos. Los que no lee no se pierden: esperan en el **búfer de recepción** del sistema operativo, y `recv` siempre entrega el más antiguo.

Fíjate en que las dos columnas cuentan historias distintas: el intervalo sigue en 500 ms, porque no falta ningún mensaje, pero la edad no para de crecer.

Preguntas

1. ¿Por qué la edad crece exactamente 1,5 s en cada línea?
2. ¿Qué valor máximo de `PROCESADO` evita que la edad crezca? Compruébalo.
3. Si este cliente mostrara la posición de un robot, ¿qué vería el operario al cabo de 10 minutos?

Experimento 2 · ¿Qué devuelve realmente `recv`?

`recv_exactly` pide exactamente 8 bytes. ¿Qué pasaría si pidiéramos «lo que haya»?

> **En el laboratorio**
>
> **Paso 5.** Detén el cliente. Escribe `e1_cliente_bytes.py` (Anexo A.3) y lánzalo. Este cliente llama a `sock.recv(1024)` (línea 15) y muestra cuántos bytes le devuelve cada llamada. Pruébalo primero con `PROCESADO = 0.0` y después con `PROCESADO = 2.0` (línea 8).

```
recv devolvió    8 bytes = 1 mensajes y 0 bytes sueltos
      mensaje 1: edad      0.7 ms
recv devolvió   32 bytes = 4 mensajes y 0 bytes sueltos
      mensaje 1: edad   1999.9 ms
      mensaje 2: edad   1499.6 ms
      mensaje 3: edad    999.3 ms
      mensaje 4: edad    499.0 ms
```

Con el cliente rápido parece que TCP entrega mensajes de 8 bytes. Con el cliente lento, una sola llamada trae todo lo que se ha acumulado.

> **Idea clave**
>
> TCP entrega un **flujo de bytes**, no mensajes (capítulo 5.2). Dónde empieza y dónde acaba cada mensaje lo decide tu programa. Por eso existe `recv_exactly`.

Preguntas

4. Este cliente lento no se va quedando atrás como el del experimento 1. ¿Por qué?
5. Imagina que `recv` devuelve 20 bytes. Mira las líneas 19–23: ¿qué hace el programa con los 4 bytes sobrantes? ¿Qué le pasaría al mensaje siguiente?

Experimento 3 · Una pausa

> **En el laboratorio**
>
> **Paso 6.** Vuelve al cliente de hora con `PROCESADO = 0.0` y lánzalo en depuración. Pon un punto de interrupción en la línea 36, cuenta 10 s con el reloj, quita el punto de interrupción y pulsa F5.

Es lo que ya viste en el paso 3. Describe lo que aparece justo al continuar: cuántos mensajes salen casi a la vez, con qué edades, y cuándo vuelve todo a la normalidad. Relaciónalo con el experimento 1.

Experimento 4 · ¿Cada cuánto envía de verdad el servidor?

Mira la columna del intervalo del paso 2: no es 500,0 ms, sino algo más.

Preguntas

6. El servidor duerme `PERIODO` segundos *después* de enviar (líneas 16–20). ¿Qué más tiempo cuenta en cada vuelta?
7. Con tu medida, ¿cuánto se habrá retrasado el servidor al cabo de una hora? (Pista: en una hora se envían 7 200 mensajes.)

Para que el error no se acumule, el servidor no debe dormir un periodo fijo, sino **hasta la hora a la que toca el siguiente envío**.

> **En el laboratorio**
>
> **Paso 7.** En el servidor, añade una línea antes del bucle de `atender` y cambia la espera del final del bucle:

```
 14      enviados = 0
         siguiente = time.monotonic()          # nueva: cuándo toca el
siguiente envío
 15      while True:
             ...
 20          siguiente += PERIODO              # sustituye a
time.sleep(PERIODO)
             time.sleep(max(0, siguiente - time.monotonic()))
```

> Relanza el servidor y el cliente y compara la columna del intervalo con la del paso 2.

`time.monotonic()` es un reloj que solo avanza: no salta si el PC cambia de hora (por ejemplo, al sincronizarse por la red). Es el adecuado para medir intervalos dentro de un mismo equipo. `time.time()` sigue siendo necesario para la hora que se envía, porque es la que el otro extremo puede comparar con la suya.

Experimento 5 · Servidor detenido frente a servidor colgado

> **En el laboratorio**
>
> **Paso 8.** Con el cliente en marcha, detén el servidor (Ctrl + C en su terminal).

El cliente termina con `Fin: El servidor cerró la conexión`. Al cerrarse el servidor, TCP avisa al otro extremo y `recv` devuelve 0 bytes (línea 17 de `recv_exactly`), como en la práctica 0.

> **En el laboratorio**
>
> **Paso 9.** Cambia los papeles: lanza el servidor con F5, en depuración, y el cliente en un terminal (`python e1_cliente_tiempo.py`). Pon un punto de interrupción en la línea 16 del servidor. Así simulas un dispositivo **colgado**: sigue conectado, pero no envía nada. Espera 20 s mirando el cliente.

Preguntas

8. ¿Qué muestra el cliente mientras el servidor está colgado? ¿Hay algún error, aviso o fin?
9. Desde el punto de vista del cliente, ¿hay alguna diferencia entre un servidor colgado y uno que tarda mucho en enviar?

> **Atención**
>
> Quédate con esta pregunta: con lo que sabes hasta ahora, un programa no tiene forma de saber que el equipo del otro lado se ha colgado. Lo resolverás en el ejemplo 3.

1.4 Ejemplo 2 · Una tarea bloqueante y la comunicación aparte

Ahora el programa tiene otra cosa que hacer además de recibir: atender a un operario. Usaremos la tarea bloqueante más fácil de ver, esperar a que pulse Enter. `input()` detiene el programa hasta que eso ocurre, igual que `recv` lo detiene hasta que llegan datos. En un programa real podría ser esperar a que el robot termine un movimiento, un cálculo largo o escribir un archivo.

El servidor es el del ejemplo 1, sin cambios. Si hiciste el paso 7, déjalo con la corrección.

Versión sin hilo

`e2_operario_sin_hilo.py` hace lo que haría cualquiera a primera vista: espera al operario y, cuando pulsa Enter, lee un mensaje.

```
 28          while True:
 29              orden = input("Pulsa Enter para ver la hora del servidor (q
para salir): ")
 30              if orden.strip().lower() == "q":
 31                  return
 32              (t_servidor,) = struct.unpack(">d", recv_exactly(sock, 8))
```

> **En el laboratorio**
>
> **Paso 10.** Con el servidor en marcha, escribe `e2_operario_sin_hilo.py` (Anexo A.4) y lánzalo **en un terminal**: `input()` necesita que puedas escribir en él. Espera unos 4 s y pulsa Enter. Espera 3 s más y pulsa Enter. Después pulsa Enter varias veces seguidas. Anota la edad que muestra cada vez.

```
Hora del servidor: 18:17:35.362   (edad   3.97 s)
Hora del servidor: 18:17:35.862   (edad   6.47 s)
Hora del servidor: 18:17:36.362   (edad   6.17 s)
```

El operario pide la hora «actual» y recibe la de hace varios segundos. Con lo que has visto en el ejemplo 1, deberías poder explicar por qué.

Preguntas

10. Mientras el programa espera en `input()`, ¿quién lee el socket? ¿Dónde están los mensajes que llegan?
11. Al pulsar Enter, ¿qué mensaje se lee: el más reciente o el más antiguo?
12. ¿Qué edad tendrá el dato si el operario tarda un minuto en pulsar Enter? ¿Y si después pulsa Enter 10 veces seguidas?

Versión con hilo

El problema es que un mismo programa no puede estar esperando en dos sitios a la vez. La solución es tener **dos hilos**. Un hilo es una segunda línea de ejecución dentro del mismo programa: avanza a la vez que el hilo principal y puede compartir datos con él.

- **Hilo receptor:** lee mensajes sin parar y guarda el último.
- **Hilo principal:** espera al operario y, cuando pulsa Enter, consulta lo último que ha guardado el receptor.

Todo lo relativo a la comunicación se encierra en una clase, `ReceptorHora` (líneas 20–69). El objeto guarda el socket, el hilo, el candado y el último dato. El programa principal no ve nada de eso: lo arranca con `start()` y le pide la hora con `leer()`.



Figura 1. Los dos hilos del ejemplo 2

Cada hilo pasa casi todo el tiempo bloqueado, pero en un sitio distinto: el receptor en `recv()` y el principal en `input()`. Solo se encuentran en el estado compartido, y siempre a través del candado.

El estado compartido (líneas 23–32). Tres atributos los escriben un hilo y los lee el otro: `_hora`, `_recibidos` y `_motivo_fin`. Por eso hay un candado, `_candado` (línea 29). El guion bajo delante del nombre es una convención de Python: indica que son internos de la clase y que desde fuera no se deben tocar.

Arrancar (líneas 34–39).

```
 36          self._sock = socket.create_connection((self.host, self.port))
 37          self._hilo = threading.Thread(target=self._recibir,
name="receptor", daemon=True)
 38          self._hilo.start()
```

Línea	Qué hace
36	Abre la conexión, como el cliente del ejemplo 1
37	Prepara un hilo que ejecutará el método `_recibir`. El nombre `receptor` es el que verás en el depurador. `daemon=True` indica que el hilo no impide que el programa termine cuando acaba el principal

Línea	Qué hace
38	Pone el hilo en marcha. Desde aquí hay dos hilos ejecutándose a la vez

Lo que hace el hilo receptor (líneas 50–60).

```
 52          try:
 53              while True:
 54                  (hora,) = struct.unpack(">d", self._recv_exactly(8))
 55                  with self._candado:
 56                      self._hora = hora
 57                      self._recibidos += 1
 58          except (ConnectionError, OSError) as exc:
 59              with self._candado:
 60                  self._motivo_fin = str(exc)
```

El hilo pasa casi todo el tiempo bloqueado en `recv`, dentro de `_recv_exactly` (línea 54). Eso ya no es un problema: no bloquea al principal. En cuanto llega un mensaje, coge el candado, actualiza el estado y lo suelta (líneas 55–57). Si la conexión se cierra, deja escrito el motivo (líneas 58–60) para que el principal se entere.

Lo que pide el hilo principal (líneas 41–44).

```
 41      def leer(self):
 42          """Devuelve una copia del último estado (lo llama el hilo
principal)."""
 43          with self._candado:
 44              return Lectura(self._hora, self._recibidos,
self._motivo_fin)
```

`leer()` coge el candado, copia los tres valores en una `Lectura` (líneas 13–17) y lo suelta. El principal trabaja después con esa copia, sin prisa y sin candado.

> **Idea clave**
>
> El candado (`threading.Lock`) solo puede tenerlo un hilo a la vez. El bloque `with self._candado:` es una **sección crítica**: mientras un hilo está dentro, el otro espera a la entrada. Así el principal nunca puede leer la hora nueva con el contador viejo. Y como el otro hilo espera, **dentro de una sección crítica nunca se espera a nada**: solo se copian o asignan valores.

> **En el laboratorio**
>
> **Paso 11.** Escribe `e2_operario_con_hilo.py` (Anexo A.5) y lánzalo con F5. Escribe en el terminal de depuración que abre VS Code. Repite lo que hiciste en el paso 10.

```
Hora del servidor: 18:17:46.427   (edad   0.43 s, 8 mensajes recibidos)
Hora del servidor: 18:17:49.429   (edad   0.43 s, 14 mensajes recibidos)
Hora del servidor: 18:17:49.929   (edad   0.13 s, 15 mensajes recibidos)
```

La edad ya no depende de lo que tarde el operario. El contador demuestra que, mientras el principal esperaba, el receptor seguía trabajando.

> **En el laboratorio**
>
> **Paso 12 · Ver los dos hilos.** Pon un punto de interrupción en la línea 56. Cuando se detenga, abre el panel **Pila de llamadas** (*Call Stack*): verás dos hilos, `MainThread` y `receptor`. Haz clic en cada uno para ver en qué línea está: uno está en `input()` y el otro, en la línea 56. Quita el punto de interrupción y continúa.

> **En el laboratorio**
>
> **Paso 13.** Detén el servidor y pulsa Enter: el programa responde `Comunicación perdida: El servidor cerró la conexión`. Después vuelve a lanzar el servidor, esta vez con F5, relanza el programa, pon un punto de interrupción en la línea 16 del servidor (servidor colgado) y pulsa Enter varias veces.

Preguntas

13. ¿Por qué con el hilo la edad nunca supera los 0,5 s? ¿Qué mensajes se pierden? ¿Importa para lo que hace este programa?
14. Sustituye `input(...)` por `time.sleep(3)`, una tarea que tarda 3 s. ¿Cambia algo en las dos versiones?
15. ¿Qué pasaría si `_recibir` hiciera el `recv` dentro del `with self._candado:`? Piensa en qué le pasaría a `leer()`.
16. Con el servidor colgado (paso 13), ¿qué te dice el programa? ¿Qué información le falta al operario?

17. ¿Qué ventaja tiene que el estado compartido viva dentro de un objeto y solo se lea con `leer()`? Piensa en qué pasaría si el programa necesitara recibir de dos servidores a la vez.

1.5 Ejemplo 3 · Un supervisor que atiende y vigila a varios dispositivos

En el experimento 5 quedó una pregunta abierta: ¿cómo sabe un programa que el equipo del otro lado se ha colgado? En este ejemplo un **supervisor** atiende a varios **dispositivos** a la vez y comprueba cada segundo cuánto hace que cada uno envió algo. Es lo que hace un controlador industrial con los equipos de campo que dependen de él.

Cambian los papeles

Ahora los dispositivos son los clientes: se conectan al supervisor (puerto **50001**) y le envían su hora cada 0,5 s, con el mismo mensaje de 8 bytes. `e3_dispositivo.py` (Anexo A.7) es casi el servidor del ejemplo 1, solo que en vez de esperar conexiones se conecta:

```
12      with socket.create_connection((HOST, PORT)) as sock:
13          ip, puerto = sock.getsockname()
14          print(f"Conectado al supervisor desde {ip}:{puerto}")
15          while True:
16              sock.sendall(struct.pack(">d", time.time()))
17              time.sleep(PERIODO)
```

`getsockname()` (línea 13) devuelve la IP y el puerto efímero de este extremo: así sabrás qué fila de la tabla del supervisor es la tuya.

Los hilos del supervisor

El supervisor tiene que hacer tres cosas que bloquean: esperar dispositivos nuevos (`accept`), recibir de cada dispositivo (`recv`, una vez por cada uno) y vigilar a todos cada segundo. Cada una va en su propio hilo.

Como en el ejemplo 2, todo vive dentro de una clase, `Supervisor` (líneas 30–121). Sus atributos son el candado y un diccionario con un objeto `Dispositivo` por cada conexión (líneas 16–27). Cada método lo ejecuta un hilo distinto:

Método	Líneas	Lo ejecuta	Dónde se bloquea
`run`	40–52	El hilo principal	`time.sleep(REVISION)`
`_aceptar`	54–62	El hilo `aceptador`	`accept()`
`_atender`	64–79	Un hilo `disp-<puerto>` por dispositivo	`recv()`
`_revisar`	81–111	El hilo principal, cada segundo	No se bloquea

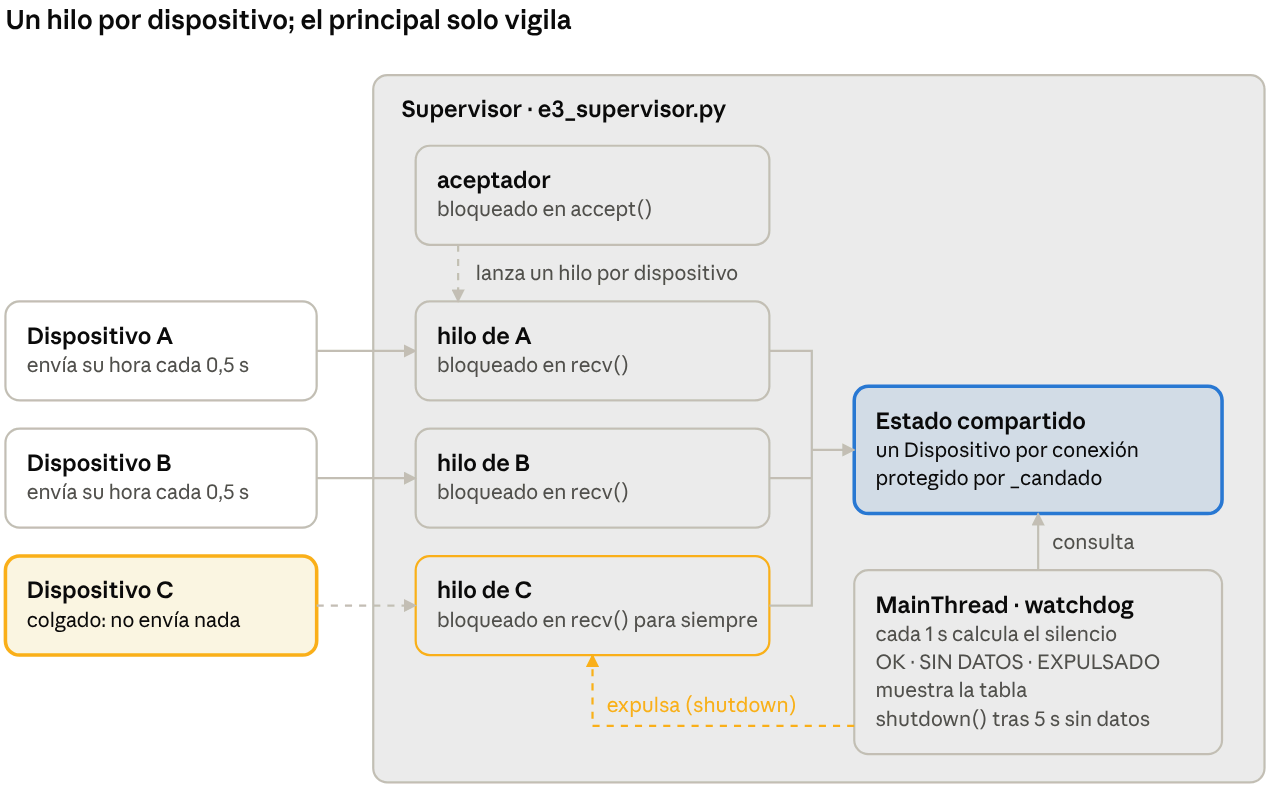


Figura 2. Los hilos del supervisor

Los hilos de cada dispositivo solo escriben en el estado compartido; el principal solo lo lee y es el único que decide expulsar.

Aceptar (líneas 54–62). Por cada conexión nueva crea un `Dispositivo`, lo añade al diccionario con el candado cogido y lanza un hilo que ejecutará `_atender` para él.

Atender (líneas 64–79). Es el receptor del ejemplo 2, uno por dispositivo. Cada vez que llega un mensaje anota cuándo llegó, con el reloj de este PC (línea 70), y el retardo respecto a la hora del dispositivo (línea 71). El bloque `finally` (líneas 75–79) se ejecuta pase lo que pase: marca el dispositivo como desconectado y cierra su socket.

El watchdog (líneas 81–111)

Cada segundo, el hilo principal calcula el **silencio** de cada dispositivo: el tiempo desde su último mensaje (línea 87). Con él lo clasifica:

Estado	Condición	Líneas	Qué significa
OK	Silencio ⩽ `WATCHDOG` (1,5 s)	100–101	Funciona
SIN DATOS	Conectado y silencio > 1,5 s	98–99	Colgado, cable cortado o muy lento: no se sabe cuál
EXPULSADO	Silencio > `EXPULSAR_TRAS` (5 s)	88–95	El supervisor da la conexión por perdida y la cierra
DESCONECTADO	El dispositivo cerró la conexión	96–97	Se ha ido de forma ordenada

`WATCHDOG = 1.5` (línea 10) son tres periodos del dispositivo: tolera que un mensaje se retrase, pero no que falten varios seguidos.

Expulsar (líneas 88–93). Un dispositivo colgado deja a su hilo bloqueado en `recv` para siempre. `sock.shutdown(socket.SHUT_RDWR)` (línea 91) cierra la conexión desde el supervisor, y eso despierta a ese hilo: su `recv` termina, entra en el `finally` y acaba limpiamente.

> **Idea clave**
>
> El silencio se mide con `time.monotonic()` **del supervisor**, no con la hora que envía el dispositivo. Lo que hay que vigilar es si *llegan* mensajes, y eso solo lo sabe quien recibe. Además, el reloj del dispositivo puede estar mal puesto o congelado con él.

> **En el laboratorio**
>
> **Paso 14.** Escribe `e3_supervisor.py` (Anexo A.6) y `e3_dispositivo.py` (Anexo A.7). Lanza el supervisor en un terminal. Abre tres terminales más y en cada uno ejecuta `python e3_dispositivo.py`. Lanza un cuarto dispositivo con F5, en depuración. Observa cómo aparecen en la tabla y cómo cambia el número de hilos.

> **En el laboratorio**
>
> **Paso 15.** En el dispositivo que está en depuración, pon un punto de interrupción en la línea 16: ese dispositivo se cuelga. En el terminal de otro dispositivo, pulsa Ctrl + C: ese se va. Observa la tabla del supervisor durante 10 s.

Con tres dispositivos, la salida es parecida a esta:

```
--- 18:18:18 · 3 dispositivos · 5 hilos ---
  127.0.0.1:34444       OK            silencio   0.1 s   recibidos     4
retardo    0.2 ms
  127.0.0.1:34448       OK            silencio   0.1 s   recibidos     4
retardo    0.2 ms
  127.0.0.1:34464       OK            silencio   0.1 s   recibidos     4
retardo    0.1 ms
--- 18:18:20 · 3 dispositivos · 4 hilos ---
  127.0.0.1:34444       OK            silencio   0.1 s   recibidos     8
retardo    0.2 ms
  127.0.0.1:34448       SIN DATOS     silencio   1.6 s   recibidos     5
retardo    0.1 ms
  127.0.0.1:34464       DESCONECTADO  silencio   0.6 s   recibidos     7
retardo    0.2 ms
--- 18:18:24 · 3 dispositivos · 4 hilos ---
  127.0.0.1:34444       OK            silencio   0.1 s   recibidos    16
retardo    0.2 ms
  127.0.0.1:34448       EXPULSADO     silencio   5.6 s   recibidos     5
retardo    0.1 ms
  127.0.0.1:34464       DESCONECTADO  silencio   4.6 s   recibidos     7
retardo    0.2 ms
--- 18:18:25 · 3 dispositivos · 3 hilos ---
```

> **En el laboratorio**
>
> **Paso 16.** Quita el punto de interrupción del dispositivo expulsado y pulsa F5. ¿Qué mensaje muestra? Relánzalo y busca su fila en la tabla.

> **Atención**
>
> No pongas puntos de interrupción en el supervisor: el depurador detiene **todos** sus hilos, incluido el watchdog. Al continuar, todos los dispositivos parecerán callados durante un instante y, si la pausa dura más de 5 s, los expulsará a todos.

Preguntas

18. Explica cómo cambia el número de hilos de la tabla en los pasos 14 y 15. ¿Qué hilos son?
19. ¿Por qué el supervisor necesita un hilo por dispositivo? ¿Qué pasaría con un solo hilo que hiciera `recv` de todos por turnos y uno se colgara?
20. ¿Qué pasaría con `WATCHDOG = 0.4`? ¿Y con `WATCHDOG = 30`? ¿De qué depende el valor adecuado?
21. ¿Por qué el dispositivo relanzado en el paso 16 aparece como uno nuevo? ¿Qué añadirías al protocolo para que se identificara siempre igual?
22. Con 1 000 dispositivos, ¿cuántos hilos tendría el supervisor? ¿Te parece razonable?
23. Vuelve al cliente del experimento 5 (servidor colgado). ¿Qué tendrías que añadirle para que avisara de que el servidor ha dejado de enviar?

1.6 Experimentos rápidos

Cada experimento dura un par de minutos. Cambia lo indicado, relanza, anota qué ves y explica la causa. Deshaz el cambio antes de pasar al siguiente.

#	Qué hacer	Programas	Qué deberías ver
1	Lanzar el cliente sin el servidor en marcha	Ejemplo 1	`Conexión rechazada: ¿está el servidor en marcha?`
2	Lanzar el servidor dos veces seguidas	Ejemplo 1	En el segundo, un error `Address already in use` (en Windows, `WinError 10048`)
3	En el servidor, `PERIODO = 0.05`, con el cliente en `PROCESADO = 2.0`	Ejemplo 1	La edad crece mucho más deprisa que en el paso 4. ¿Cuánto por línea?

#	Qué hacer	Programas	Qué deberías ver
4	En el servidor, cambiar `">d"` por `">f"` en la línea 16	Ejemplo 1	El cliente termina con un error al convertir la hora: ahora cada lectura de 8 bytes junta dos mensajes de 4
5	En el cliente, cambiar `">d"` por `"<d"` en la línea 34	Ejemplo 1	El mismo error: los 8 bytes son correctos, pero se leen en el orden equivocado (como el experimento 5 de la práctica 0)
6	En el supervisor, `WATCHDOG = 0.4`	Ejemplo 3	Dispositivos sanos que pasan a SIN DATOS de vez en cuando
7	En un dispositivo, `PERIODO = 2.0`	Ejemplo 3	Ese dispositivo alterna entre OK y SIN DATOS, aunque funciona

1.7 Ampliación: dos PC del laboratorio

Hasta ahora todo ocurría dentro de tu PC. Con dos cambios, los mismos programas funcionan entre equipos, como en la ampliación de la práctica 0. Poneos de acuerdo por parejas.

> **En el laboratorio**
>
> **Paso 17.** Uno de los dos lanza el servidor del ejemplo 1 con `HOST = "0.0.0.0"` (línea 7). El otro pone en el cliente la IP del primero (línea 7) y lo lanza. Comprobad antes con `ping` que hay conexión.

Fíjate en la edad del dato. Ya no son décimas de milisegundo: puede ser de varios milisegundos, de segundos, o incluso **negativa**. La edad compara la hora de dos relojes distintos, así que mezcla lo que tarda el mensaje en viajar con la diferencia entre los relojes de los dos PC.

> **En el laboratorio**
>
> **Paso 18.** Repetid el ejemplo 3 entre equipos: el supervisor en un PC (`HOST = "0.0.0.0"`) y los dispositivos en los de los compañeros. Desconectad el cable de red de uno de los dispositivos y mirad su fila en la tabla.

> **Atención**
>
> Como en la práctica 0, la primera vez que un programa escucha en la red el cortafuegos de Windows puede pedir permiso: acepta para redes privadas. Si el cliente da `TimeoutError`, consúltalo con el profesor.

Preguntas

24. En el paso 17, ¿cómo distinguirías la parte de la edad que se debe a la red de la que se debe a los relojes?
25. ¿Qué haría falta para que la edad del dato fuera fiable entre dos equipos?
26. En el paso 18, ¿en qué estado aparece el dispositivo desconectado? ¿Por qué no aparece como DESCONECTADO?

Anexo A · Listados completos

Los siete programas de la práctica, con los números de línea que se citan en el enunciado. Escríbelos a mano, cada uno cuando lo pida su paso, **sin los números de línea** ni los dos espacios que los separan del código. Respeta la sangría: en Python marca qué líneas forman parte de cada bloque.

A.1 e1_servidor_tiempo.py

```
  1  """Ejemplo 1: servidor que envía la hora (8 bytes, ">d") cada PERIODO
segundos."""
  2  import socket
  3  import struct
  4  import sys
  5  import time
  6
  7  HOST = "127.0.0.1"   # "0.0.0.0" para aceptar conexiones de otros PC
  8  PORT = 50000
  9  PERIODO = 0.5        # segundos entre mensajes
 10
 11
 12  def atender(conexion):
 13      """Envía la hora al cliente cada PERIODO segundos hasta que se
desconecte."""
 14      enviados = 0
 15      while True:
```

```
 16          conexion.sendall(struct.pack(">d", time.time()))
 17          enviados += 1
 18          if enviados % 10 == 0:
 19              print(f"{enviados} mensajes enviados")
 20          time.sleep(PERIODO)
 21
 22
 23  def main():
 24      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as escucha:
 25          if sys.platform != "win32":
 26              escucha.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
1)
 27          escucha.bind((HOST, PORT))
 28          escucha.listen(1)
 29          while True:
 30              print(f"Esperando un cliente en {HOST}:{PORT} ...")
 31              conexion, direccion = escucha.accept()
 32              print(f"Cliente conectado desde {direccion[0]}:
{direccion[1]}")
 33              with conexion:
 34                  try:
 35                      atender(conexion)
 36                  except (BrokenPipeError, ConnectionResetError,
ConnectionAbortedError):
 37                      print("El cliente se ha desconectado")
 38
 39
 40  if __name__ == "__main__":
 41      try:
 42          main()
 43      except KeyboardInterrupt:
 44          print("\nServidor detenido.")
```

A.2 e1_cliente_tiempo.py

```
  1  """Ejemplo 1: cliente que recibe la hora y muestra su edad y el
intervalo."""
  2  import datetime
  3  import socket
  4  import struct
  5  import time
  6
```

```
  7  HOST = "127.0.0.1"
  8  PORT = 50000
  9  PROCESADO = 0.0    # segundos que «tarda» el programa con cada dato
 10
 11
 12  def recv_exactly(sock, size):
 13      """Recibe exactamente size bytes (igual que en la práctica 0)."""
 14      datos = bytearray()
 15      while len(datos) < size:
 16          trozo = sock.recv(size - len(datos))
 17          if not trozo:
 18              raise ConnectionError("El servidor cerró la conexión")
 19          datos.extend(trozo)
 20      return bytes(datos)
 21
 22
 23  def hora_legible(t):
 24      """Convierte segundos desde 1970 en hh:mm:ss.mmm."""
 25      return datetime.datetime.fromtimestamp(t).strftime("%H:%M:%S.%f")
[:-3]
 26
 27
 28  def main():
 29      with socket.create_connection((HOST, PORT)) as sock:
 30          print(f"Conectado a {HOST}:{PORT}")
 31          anterior = None
 32          n = 0
 33          while True:
 34              (t_servidor,) = struct.unpack(">d", recv_exactly(sock, 8))
 35              n += 1
 36              edad = time.time() - t_servidor
 37              texto = f"n = {n:4d}   hora del servidor
{hora_legible(t_servidor)}"
 38              texto += f"   edad {edad * 1000:8.1f} ms"
 39              if anterior is not None:
 40                  texto += f"   intervalo {(t_servidor - anterior) *
1000:7.1f} ms"
 41              print(texto)
 42              anterior = t_servidor
 43              time.sleep(PROCESADO)
 44
 45
 46  if __name__ == "__main__":
```

```
 47      try:
 48          main()
 49      except KeyboardInterrupt:
 50          print("\nInterrumpido por el usuario.")
 51      except ConnectionRefusedError:
 52          print("Conexión rechazada: ¿está el servidor en marcha?")
 53      except ConnectionError as exc:
 54          print(f"Fin: {exc}")
```

A.3 e1_cliente_bytes.py

```
  1  """Ejemplo 1: muestra cuántos bytes devuelve cada llamada a
recv(1024)."""
  2  import socket
  3  import struct
  4  import time
  5
  6  HOST = "127.0.0.1"
  7  PORT = 50000
  8  PROCESADO = 0.0
  9
 10
 11  def main():
 12      with socket.create_connection((HOST, PORT)) as sock:
 13          print(f"Conectado a {HOST}:{PORT}")
 14          while True:
 15              datos = sock.recv(1024)
 16              if not datos:
 17                  print("Fin: el servidor cerró la conexión")
 18                  return
 19              completos, sobran = divmod(len(datos), 8)
 20              print(f"recv devolvió {len(datos):4d} bytes = {completos}
mensajes"
 21                    f" y {sobran} bytes sueltos")
 22              for i in range(completos):
 23                  (t,) = struct.unpack_from(">d", datos, 8 * i)
 24                  print(f"      mensaje {i + 1}: edad {(time.time() - t) *
1000:8.1f} ms")
 25              time.sleep(PROCESADO)
 26
 27
 28  if __name__ == "__main__":
```

```
    try:
        main()
    except KeyboardInterrupt:
        print("\nInterrumpido por el usuario.")
```

A.4 e2_operario_sin_hilo.py

```
"""Ejemplo 2a: la tarea bloqueante (input) y la comunicación en el mismo hilo."""
import datetime
import socket
import struct
import time

HOST = "127.0.0.1"
PORT = 50000


def recv_exactly(sock, size):
    datos = bytearray()
    while len(datos) < size:
        trozo = sock.recv(size - len(datos))
        if not trozo:
            raise ConnectionError("El servidor cerró la conexión")
        datos.extend(trozo)
    return bytes(datos)


def hora_legible(t):
    return datetime.datetime.fromtimestamp(t).strftime("%H:%M:%S.%f")[:-3]


def main():
    with socket.create_connection((HOST, PORT)) as sock:
        print(f"Conectado a {HOST}:{PORT}")
        while True:
            orden = input("Pulsa Enter para ver la hora del servidor (q para salir): ")
            if orden.strip().lower() == "q":
                return
            (t_servidor,) = struct.unpack(">d", recv_exactly(sock, 8))
```

```
            print(f"   Hora del servidor: {hora_legible(t_servidor)}"
                  f"   (edad {time.time() - t_servidor:6.2f} s)")


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\nInterrumpido por el usuario.")
    except ConnectionError as exc:
        print(f"Fin: {exc}")
```

A.5 e2_operario_con_hilo.py

```
"""Ejemplo 2b: la comunicación en un hilo aparte, dentro de la clase ReceptorHora."""
import datetime
import socket
import struct
import threading
import time
from typing import NamedTuple, Optional

HOST = "127.0.0.1"
PORT = 50000


class Lectura(NamedTuple):
    """Copia del estado del receptor en un instante dado."""
    hora: Optional[float]         # hora del servidor en el último mensaje
    recibidos: int                # mensajes recibidos en total
    motivo_fin: Optional[str]     # por qué terminó el receptor (None si funciona)


class ReceptorHora:
    """Recibe en segundo plano la hora que envía el servidor y guarda la última."""

    def __init__(self, host, port):
        self.host = host
```

```
 25          self.port = port
 26          self._sock = None
 27          self._hilo = None
 28          # Estado compartido entre los dos hilos: solo se toca con el
candado cogido.
 29          self._candado = threading.Lock()
 30          self._hora = None
 31          self._recibidos = 0
 32          self._motivo_fin = None
 33
 34      def start(self):
 35          """Abre la conexión y lanza el hilo receptor (lo llama el hilo
principal)."""
 36          self._sock = socket.create_connection((self.host, self.port))
 37          self._hilo = threading.Thread(target=self._recibir,
name="receptor", daemon=True)
 38          self._hilo.start()
 39          return self
 40
 41      def leer(self):
 42          """Devuelve una copia del último estado (lo llama el hilo
principal)."""
 43          with self._candado:
 44              return Lectura(self._hora, self._recibidos,
self._motivo_fin)
 45
 46      def close(self):
 47          if self._sock is not None:
 48              self._sock.close()
 49
 50      def _recibir(self):
 51          """Cuerpo del hilo receptor: lee sin parar y guarda el último
dato."""
 52          try:
 53              while True:
 54                  (hora,) = struct.unpack(">d", self._recv_exactly(8))
 55                  with self._candado:
 56                      self._hora = hora
 57                      self._recibidos += 1
 58          except (ConnectionError, OSError) as exc:
 59              with self._candado:
 60                  self._motivo_fin = str(exc)
 61
```

```
 62      def _recv_exactly(self, size):
 63          datos = bytearray()
 64          while len(datos) < size:
 65              trozo = self._sock.recv(size - len(datos))
 66              if not trozo:
 67                  raise ConnectionError("El servidor cerró la conexión")
 68              datos.extend(trozo)
 69          return bytes(datos)
 70
 71
 72  def hora_legible(t):
 73      return datetime.datetime.fromtimestamp(t).strftime("%H:%M:%S.%f")
[:-3]
 74
 75
 76  def main():
 77      receptor = ReceptorHora(HOST, PORT).start()
 78      print(f"Conectado a {HOST}:{PORT}")
 79      try:
 80          while True:
 81              orden = input("Pulsa Enter para ver la hora del servidor (q
para salir): ")
 82              if orden.strip().lower() == "q":
 83                  return
 84              lectura = receptor.leer()
 85              if lectura.motivo_fin is not None:
 86                  print(f"   Comunicación perdida: {lectura.motivo_fin}")
 87              elif lectura.hora is None:
 88                  print("   Todavía no ha llegado ningún mensaje")
 89              else:
 90                  print(f"   Hora del servidor:
{hora_legible(lectura.hora)}"
 91                        f"   (edad {time.time() - lectura.hora:6.2f} s,"
 92                        f" {lectura.recibidos} mensajes recibidos)")
 93      finally:
 94          receptor.close()
 95
 96
 97  if __name__ == "__main__":
 98      try:
 99          main()
100      except KeyboardInterrupt:
101          print("\nInterrumpido por el usuario.")
```

A.6 e3_supervisor.py

```
  1  """Ejemplo 3: supervisor que atiende a varios dispositivos y los vigila
(watchdog)."""
  2  import socket
  3  import struct
  4  import sys
  5  import threading
  6  import time
  7
  8  HOST = "127.0.0.1"     # "0.0.0.0" para aceptar dispositivos de otros PC
  9  PORT = 50001
 10  WATCHDOG = 1.5         # s sin datos para pasar a SIN DATOS (3 periodos)
 11  EXPULSAR_TRAS = 5.0    # s sin datos para cerrar la conexión
 12  OLVIDAR_TRAS = 10.0    # s que se sigue mostrando un dispositivo que ya
no está
 13  REVISION = 1.0         # s entre revisiones del watchdog
 14
 15
 16  class Dispositivo:
 17      """Lo que el supervisor sabe de un dispositivo conectado."""
 18
 19      def __init__(self, sock, direccion):
 20          self.sock = sock
 21          self.direccion = direccion
 22          self.conectado = True
 23          self.expulsado = False
 24          self.ultima_recepcion = time.monotonic()   # reloj local: para
el watchdog
 25          self.retardo = None                        # hora local − hora
del dispositivo
 26          self.recibidos = 0
 27          self.momento_fin = None
 28
 29
 30  class Supervisor:
 31      """Atiende a cada dispositivo en su hilo y los vigila desde el hilo
principal."""
 32
 33      def __init__(self, host, port):
 34          self.host = host
 35          self.port = port
 36          # Estado compartido por todos los hilos: solo se toca con el
```

```
candado cogido.
 37          self._candado = threading.Lock()
 38          self._dispositivos = {}    # dirección -> Dispositivo
 39
 40      def run(self):
 41          """Hilo principal: abre el puerto, lanza el aceptador y revisa
cada segundo."""
 42          with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
escucha:
 43              if sys.platform != "win32":
 44                  escucha.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
 45              escucha.bind((self.host, self.port))
 46              escucha.listen()
 47              print(f"Supervisor esperando dispositivos en {self.host}:
{self.port}")
 48              threading.Thread(target=self._aceptar, args=(escucha,),
 49                               name="aceptador", daemon=True).start()
 50              while True:
 51                  time.sleep(REVISION)
 52                  self._revisar()
 53
 54      def _aceptar(self, escucha):
 55          """Hilo aceptador: espera dispositivos nuevos y lanza un hilo
para cada uno."""
 56          while True:
 57              sock, (ip, puerto) = escucha.accept()
 58              d = Dispositivo(sock, f"{ip}:{puerto}")
 59              with self._candado:
 60                  self._dispositivos[d.direccion] = d
 61              threading.Thread(target=self._atender, args=(d,),
 62                               name=f"disp-{puerto}", daemon=True).start()
 63
 64      def _atender(self, d):
 65          """Un hilo por dispositivo: recibe hasta que la conexión se
cierra."""
 66          try:
 67              while True:
 68                  (t_dispositivo,) = struct.unpack(">d",
self._recv_exactly(d.sock, 8))
 69                  with self._candado:
 70                      d.ultima_recepcion = time.monotonic()
 71                      d.retardo = time.time() - t_dispositivo
```

```
 72                     d.recibidos += 1
 73          except (ConnectionError, OSError):
 74              pass
 75          finally:
 76              with self._candado:
 77                  d.conectado = False
 78                  d.momento_fin = time.monotonic()
 79              d.sock.close()
 80
 81      def _revisar(self):
 82          """Watchdog: clasifica a cada dispositivo, expulsa a los mudos y
muestra la tabla."""
 83          ahora = time.monotonic()
 84          filas = []
 85          with self._candado:
 86              for direccion, d in list(self._dispositivos.items()):
 87                  silencio = ahora - d.ultima_recepcion
 88                  if d.conectado and silencio > EXPULSAR_TRAS:
 89                      d.expulsado = True
 90                      try:
 91                          d.sock.shutdown(socket.SHUT_RDWR)   # despierta
a su hilo
 92                      except OSError:
 93                          pass
 94                  if d.expulsado:
 95                      estado = "EXPULSADO"
 96                  elif not d.conectado:
 97                      estado = "DESCONECTADO"
 98                  elif silencio > WATCHDOG:
 99                      estado = "SIN DATOS"
100                  else:
101                      estado = "OK"
102                  if not d.conectado and ahora - d.momento_fin >
OLVIDAR_TRAS:
103                      del self._dispositivos[direccion]
104                      continue
105                  retardo = "     —" if d.retardo is None else f"
{d.retardo * 1000:6.1f}"
106                  filas.append(f"  {direccion:<21} {estado:<13} silencio
{silencio:5.1f} s"
107                               f"   recibidos {d.recibidos:5d}   retardo
{retardo} ms")
108          print(f"--- {time.strftime('%H:%M:%S')} · {len(filas)}
```

```
dispositivos"
109                 f" · {threading.active_count()} hilos ---")
110           for fila in filas:
111               print(fila)
112
113       @staticmethod
114       def _recv_exactly(sock, size):
115           datos = bytearray()
116           while len(datos) < size:
117               trozo = sock.recv(size - len(datos))
118               if not trozo:
119                   raise ConnectionError("conexión cerrada")
120               datos.extend(trozo)
121           return bytes(datos)
122
123
124   if __name__ == "__main__":
125       try:
126           Supervisor(HOST, PORT).run()
127       except KeyboardInterrupt:
128           print("\nSupervisor detenido.")
```

A.7 e3_dispositivo.py

```
  1  """Ejemplo 3: dispositivo que envía su hora al supervisor cada PERIODO
segundos."""
  2  import socket
  3  import struct
  4  import time
  5
  6  HOST = "127.0.0.1"   # IP del supervisor
  7  PORT = 50001
  8  PERIODO = 0.5
  9
 10
 11  def main():
 12      with socket.create_connection((HOST, PORT)) as sock:
 13          ip, puerto = sock.getsockname()
 14          print(f"Conectado al supervisor desde {ip}:{puerto}")
 15          while True:
 16              sock.sendall(struct.pack(">d", time.time()))
 17              time.sleep(PERIODO)
```

```
 18
 19
 20  if __name__ == "__main__":
 21      try:
 22          main()
 23      except KeyboardInterrupt:
 24          print("\nDispositivo detenido.")
 25      except (BrokenPipeError, ConnectionResetError,
ConnectionAbortedError):
 26          print("El supervisor ha cerrado la conexión")
```